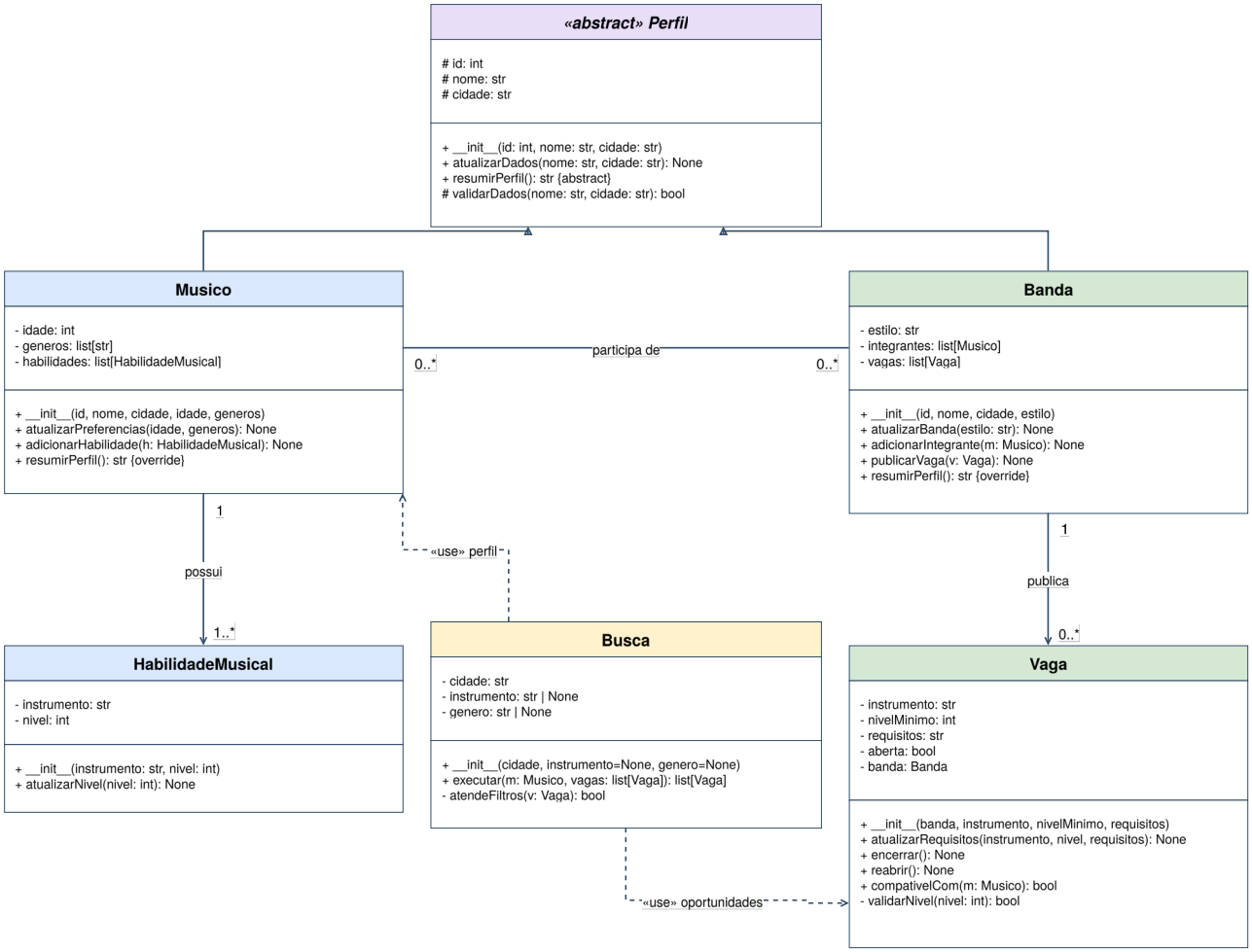
Diagrama UML – Grupo 16

ETAPA 2 · DIAGRAMA DE CLASSES

Grupo 16 | Conexão entre músicos e bandas locais | Python + Tkinter



NOTAÇÃO UML
+ público – privado # protegido
Triângulo vazio: herança, apontando para Perfil.
Linha contínua: associação; pontilhada: dependência de uso.
0..* = zero ou mais · 1..* = um ou mais · 1 = exatamente um.

REGRAS DO MODELO
Níveis: 1 = iniciante; 2 = intermediário; 3 = avançado.
Compatibilidade: vaga aberta + instrumento + nível mínimo.
Busca também aplica cidade, instrumento e gênero escolhidos.
resumirPerfil() é polimórfico: versão de músico ou de banda.

DICIONÁRIO DAS CLASSES

Responsabilidades, atributos e métodos | O detalhamento complementa a página 01

Perfil · base abstrata

Propósito: compartilhar identidade e localização dos perfis.
id: identificador único do cadastro. **nome:** identificação exibida.
cidade: município do músico ou da banda.

__init__: inicializa os dados comuns.
atualizarDados: altera nome e cidade após validação.
resumirPerfil: contrato abstrato para a apresentação do perfil.
validarDados: verifica nome e cidade não vazios; uso protegido.

Musico e Banda herdam esses atributos e métodos.

Musico · perfil do musicista

Propósito: representar quem procura uma banda.
idade: idade em anos. **generos:** estilos musicais de interesse.
habilidades: instrumentos e níveis de domínio do músico.

__init__: inicializa o perfil e as preferências; cria lista de habilidades.
atualizarPreferencias: altera idade e gêneros informados.
adicionarHabilidade: associa um instrumento e nível ao músico.
resumirPerfil: retorna nome, cidade, idade e habilidades.

Para participar da busca, deve ter ao menos uma habilidade.

Banda · perfil do grupo musical

Propósito: representar uma banda que procura integrantes.
estilo: gênero musical principal.
integrantes: músicos atuais. **vagas:** oportunidades publicadas.

__init__: inicializa dados da banda e listas de integrantes/vagas.
atualizarBanda: altera o estilo musical.
adicionarIntegrante: registra um músico na formação.
publicarVaga: associa uma vaga à banda responsável.
resumirPerfil: retorna nome, cidade, estilo e formação.

HabilidadeMusical · instrumento e domínio

Propósito: descrever uma habilidade de um músico.
instrumento: especialidade, incluindo voz/vocal.
nivel: 1 (iniciante), 2 (intermediário) ou 3 (avançado).

__init__: recebe instrumento e nível válido.
atualizarNivel: registra a evolução de habilidade, validando 1 a 3.

Uma habilidade pertence a exatamente um músico.
Um músico pode possuir várias habilidades.
Os atributos privados são manipulados pela própria classe.

Vaga · oportunidade de participação

Propósito: registrar a necessidade de uma banda.
instrumento: especialidade procurada. **nivelMinimo:** domínio exigido.
requisitos: condições em texto livre, exibidas ao músico.
aberta: disponibilidade. **banda:** referência à banda responsável.

__init__: cria vaga aberta e vincula à banda.
atualizarRequisitos: altera instrumento, nível e condições.
encerrar / reabrir: tornam a vaga indisponível / disponível.
compativelCom: confere vaga aberta, instrumento e nível do músico.
validarNivel: método privado que aceita apenas níveis de 1 a 3.

Busca · seleção de oportunidades

Propósito: encontrar vagas compatíveis com perfil e filtros.
cidade: município desejado; padrão = cidade do músico.
instrumento: filtro opcional de especialidade.
genero: filtro opcional de estilo da banda.

__init__: configura cidade e filtros opcionais.
executar: recebe músico e vagas; retorna as vagas compatíveis.
atendeFiltros: método privado que testa cidade, instrumento e gênero.

Requisitos em texto livre exigem avaliação humana.
Não há cálculo de distância entre cidades nesta proposta.

Polimorfismo: para uma lista de objetos Perfil, a mesma chamada resumirPerfil() usa a implementação de Musico ou Banda.
Encapsulamento em Python: # indica convenção _atributo; – indica __atributo. Acesso externo por métodos/propriedades públicas.
Tipos: str = texto; int = inteiro; bool = lógico; list[T] = lista; None = ausência de retorno/valor opcional.

ENTRADAS, SAÍDAS E INTERAÇÃO

Grupo 16 · Proposta de funcionamento em Python + Tkinter

01 · ENTRADA POR FORMULÁRIOS

Músico: nome, idade, cidade, instrumentos, nível e gêneros.

Banda: nome, cidade, estilo e seleção dos integrantes.

Vaga: banda, instrumento, nível mínimo e requisitos.

Busca: músico selecionado, cidade, instrumento e gênero.

02 · PROCESSAMENTO

Validar campos obrigatórios, idade inteira não negativa e nível entre 1 e 3.

Criar ou atualizar os objetos cadastrados.

Busca.executar() avalia vagas abertas e habilidades do músico.

Aplicar cidade e filtros opcionais; requisitos livres são exibidos para leitura.

03 · SAÍDA NA TELA

Confirmação de cadastro e erros de validação.

Resumo do perfil do músico e suas habilidades.

Tabela com banda, cidade, estilo, instrumento e nível exigido.

Detalhes: integrantes, requisitos e situação da vaga.

Mensagem quando não houver resultados.

MENU E TELAS PROPOSTAS

Menu: Músicos | Bandas | Vagas | Buscar oportunidades

Cadastros: campos de texto, listas de instrumentos/gêneros e seletor de nível; botões Novo, Salvar e Editar.

Vagas: selecionar banda, cadastrar/editar requisitos e Encerrar ou Reabrir.

Busca: selecionar perfil, ajustar filtros, clicar Buscar e escolher um resultado para Ver detalhes.

ARQUIVOS E PERSISTÊNCIA · DECISÃO PROPOSTA

Sim: carregar e salvar um arquivo local **dados.json** com músicos, habilidades, bandas e vagas.
IDs preservam referências entre bandas e músicos e entre vagas e bandas.
Carregar ao abrir; salvar após cadastro, edição ou mudança de situação da vaga.
Busca é temporária: filtros e resultados não precisam ser persistidos.
Não há importação de arquivos pelo usuário; a saída inicial é uma tabela no Tkinter, sem PDF ou gráficos.